ChatGPT

Planner-First Architecture for AI Lab and MutesHand

Decision summary

The best fit for **AI Lab / MutesHand** is a **single-runtime-agent, planner-first architecture** with a **deterministic control plane** around it. In practice, that means keeping **AG1** as the only active runtime agent, letting it produce and revise plans, but forcing all real-world execution through `system_entry`, a deterministic executor, approval gates, and validation rules. This aligns with Anthropic's distinction between **workflows** for predictable, code-shaped control paths and **agents** for dynamic tool-using loops; it also matches OpenAI's "manager stays in control" pattern, where specialists should remain bounded capabilities unless ownership of the response truly needs to transfer. LangGraph's plan-and-execute family supports this split between planning and execution, while MCP's separation of **resources**, **prompts**, and **tools** provides a clean contract boundary for local data, reusable prompts, and side-effectful actions. [1]

For your constraints, I do **not** recommend jumping to a multi-agent architecture now. Anthropic's guidance is to start with the simplest solution that works, and OpenAI's orchestration guidance similarly says to add specialists only when the contract materially changes. Anthropic's own multi-agent research system shows why multi-agent systems can outperform on broad, highly parallel research tasks, but it also documents higher coordination complexity, more token burn, and new reliability problems. That makes multi-agent fan-out an optimization for a later stage, not the foundation of a local mixed-workflow assistant. [2]

The target architecture should therefore have five layers. **Control plane:** request normalization, lifecycle state machine, policy engine, budget accounting, approvals, and deterministic `execution_result` assembly. **Cognitive plane:** AG1 planning, replanning, and step-wise tool selection under constrained profiles. **Execution plane:** local tools, MCP adapters, sandboxed compute, and deterministic file/document/tabular services. **Evidence plane:** all retrieved facts, rows, chunks, pages, and web snippets become addressable evidence objects with provenance. **Projection plane:** GUI and frontend subscribe to traces and state changes, but never become the source of truth. That mirrors the "harness vs compute" separation OpenAI recommends for agent systems that manipulate files and tools, where orchestration, approvals, traceability, and recovery stay outside the mutable execution workspace. [3]

What the major frameworks imply

ReAct is still the right primitive for local autonomy at the step level: it interleaves reasoning and actions so the model can update plans from real tool observations instead of relying on one-shot speculation. The original paper's key point is that reasoning traces help the model induce and update plans while actions gather new information from the environment. But pure ReAct loops also accumulate tool history turn after turn, and LangChain's own discussion of "action agents" notes that growing objectives and reliability instructions make prompt size and control harder over time. In your system, that means ReAct belongs **inside** the executor loop for a bounded step, not as the only global orchestration pattern. [4]

LangGraph plan-and-execute is more directly aligned with your needs. LangChain's planning-agent guidance argues that separating a planner from executors can be faster, cheaper, and more reliable than consulting the large planner model after every tool action. Their newer planning patterns also add two ideas that are especially relevant here: **replanning after execution** and **variable-based step references** so later steps can depend on earlier outputs without forcing a full replan every time. Those ideas map well to a local plan IR: steps should have stable IDs, typed outputs, dependency edges, and a controlled replan boundary after new evidence arrives. [5]

OpenAI's Agents SDK is useful here not because you should reproduce it literally, but because it identifies the right primitives for a production orchestration system: **tools**, **sessions**, **guardrails**, **tracing**, and the distinction between **handoffs** and **agents-as-tools**. The most relevant lesson for your design is that bounded helpers should stay behind a stable manager when the manager should own the final answer. That strongly supports your constraint that AG1 remains the active runtime selector and that "specialization" should usually be implemented as profiles, routines, or tool namespaces, not autonomous co-equal agents. The SDK's separation between conversation history and local run context is also important for privacy: the runtime may need local state and credentials that the model should not see. [6]

Anthropic's effective-agents guidance supplies the architectural north star: use the simplest composition that solves the task; treat routing, evaluator-optimizer loops, and orchestrator-worker patterns as composable patterns rather than identities; and invest heavily in the agent-computer interface by keeping tools minimal, legible, and well documented. For your use case, that implies a system that mixes workflow-style determinism with agent-style local autonomy: deterministic lifecycle and governance, but dynamic plan revision and tool use where the task is genuinely open-ended. Anthropic's later writing on context engineering strengthens that conclusion by arguing for **just-in-time context acquisition** using references, lightweight identifiers, and targeted retrieval rather than dumping large blobs of spreadsheet or document content into the model at once. [7]

MCP is especially relevant because your assistant is local-first. Its specification makes a clean distinction among **resources** for context/data, **prompts** for reusable message templates/workflows, and **tools** for executable functions. That is the right conceptual boundary for AI Lab / MutesHand: spreadsheets, documents, parsed page ranges, and web snapshots should usually be exposed as **resources**; reusable planning or extraction recipes as **prompts**; and side-effecting actions such as writes, network calls, and shell/code execution as **tools**. MCP's security model also explicitly emphasizes user consent, control, visible tool invocation, and caution around arbitrary tool behavior, which fits your approval-gated local-first requirement. [8]

LlamaIndex contributes two practical design ideas you should adopt even if you do not use the library directly. First, its workflows model is event-driven and step-based, which is a good fit for deterministic orchestration of planner output. Second, its retrieval stack highlights citation-first answering, recursive retrieval, and reference-based traversal of structured or hierarchical sources. Those patterns are directly applicable to document Q&A, small-to-big retrieval, sheet/row resolution, and corrective retrieval loops that escalate to web search only after evaluating whether local retrieval was sufficient. [9]

A brief note on **other modern frameworks**: Microsoft's Agent Framework now positions itself as the successor to AutoGen and explicitly says to use **workflows** when the process has well-defined steps and **agents** when the task is open-ended. AutoGen itself is now in maintenance mode. That is further evidence that your architecture should not default to multi-agent composition. It should default to **one**

orchestrating agent with deterministic workflows around it, then add more autonomy only where the work is truly open-ended and parallelizable. [10]

Recommended target architecture

The core recommendation is a **compiled-plan architecture**. AG1 receives the user request plus normalized workspace state and produces a **Plan IR**, not direct tool calls. `system_entry` then validates that plan against governance rules, compiles the next executable step into a **step-scoped capability profile**, executes it deterministically, stores the outputs as evidence/artifacts, runs validators, and only then returns control to AG1 for continuation or replanning. This preserves your requirement that AG1 is the only active runtime tool-selection agent while also keeping lifecycle, governance, and `execution_result` deterministic. LangChain's planning patterns, OpenAI's harness/compute split, and LlamaIndex's validated event graph all point toward this separation between planning intent and execution authority. [11]

The **step-scoped capability profile** should be the main control mechanism between planner and runtime. Instead of exposing the full tool surface on every turn, each step gets a profile such as `document_qa`, `tabular_lookup`, `tabular_analysis`, `web_research`, `compute_and_write`, or `final_synthesis`. Each profile contains the allowed tool namespaces, output schema, evidence requirements, budget limits, approval policy, privacy policy, and retry/validation rules. This mirrors OpenAI's tool-search and namespace guidance: the model performs better when the searchable surface is well organized, high-level, and not bloated with overlapping tools. Anthropic's tool-design guidance arrives at the same conclusion from another angle: overlapping, ambiguous tools are a common failure mode, and the tool interface deserves as much care as a human-computer interface. [12]

The **foundational tools** should be intentionally small in number and broad in utility. A strong starting set is: local file/resource listing and reading; spreadsheet inspection and range reading; document parsing and page/chunk retrieval; local structured query/aggregation; safe compute in a sandbox; web search/open/extract; artifact write/replace; approval request; and trace/evidence persistence. These should be grouped into namespaces or MCP servers such as `fs`, `tabular`, `documents`, `compute`, `web`, `artifacts`, and `governance`. OpenAI recommends namespaces or MCP servers for tool search and suggests keeping namespaces relatively small and clear; MCP itself makes discovery, listing, reading, and invocation first-class protocol operations. [13]

The **foundational capabilities** should be one level higher than tools and should be what the planner reasons about. A minimal capability set is: reference resolution; source acquisition; document retrieval and citation; row/entity lookup; bounded structured analysis; computation/transformation; artifact writing; grounding validation; and final synthesis. This mirrors Anthropic's "augmented LLM" view, where retrieval, tools, and memory are the building blocks, and aligns with LlamaIndex's separation among retrieval, postprocessing, synthesis, and workflow control. In other words, the planner should say "perform bounded structured analysis on the resolved table" rather than "call `read_xlsx`, then `scan_headers`, then `duckdb_query`." [14]

One important architectural choice is how to represent **memory and state**. Use three layers. **Conversation state** is the user-visible thread. **Run state** is the authoritative execution state for one request, including plan version, step statuses, evidence graph, approvals, budgets, and artifacts. **Workspace state** is the local mutable environment such as files, notebooks, temporary tables, and output documents. OpenAI's sessions

documentation and sandbox guidance are useful analogies here: sessions preserve conversational memory, while the execution workspace is a different layer with its own lifecycle and persistence concerns. [15]

Planner loop and resolution model

The planner loop should be **hybrid**, not purely upfront and not purely reactive. The recommended sequence is: **draft an initial coarse plan**, **resolve missing references**, **execute the next bounded step**, **validate**, **update notes/evidence**, and **replan only when acquisition changes the feasible path or a validator fails**. This takes the best parts of plan-and-execute, ReAct, and corrective retrieval. LangChain's planning patterns explicitly support replanning after execution; ReAct shows the value of interleaving action with reasoning; and LlamaIndex's query-planning and corrective-RAG workflows show how an evaluation step can decide whether to continue locally, escalate to search, or finalize. [16]

That leads to a specific answer for **upfront planning vs staged planning vs replan-after-acquisition**. Use **upfront planning** only for the top-level decomposition and user-visible intent summary. Use **staged planning** for step compilation, where AG1 decides the next concrete action under a restricted profile. Use **replan-after-acquisition** whenever a step returns new external knowledge, resolves a previously ambiguous reference, or fails validation. This is better than a fixed up-front mega-plan because mixed workflows often contain unknowns: the relevant worksheet might not be known yet, the correct row might be ambiguous, a document answer may require section-level retrieval, or the first web search may reveal a better avenue. Anthropic's guidance is explicit that agents are useful when you cannot hardcode the required number of steps and must adapt to environmental feedback; their research-system post likewise describes a lead agent that synthesizes findings and decides whether more research is needed. [17]

A good **Plan IR** should therefore be explicit and typed. Each step should include: `step_id`, `goal`, `kind`, `inputs`, `depends_on`, `profile`, `expected_output_schema`, `budget`, `approval_requirement`, `evidence_requirement`, and `success_predicate`. Use variable references for outputs from prior steps, in the spirit of the `#E1` / `#E2` style LangChain describes for planning agents. That makes later steps refer to resolved rows, extracted entities, or search result sets without copying bulky content into every prompt. It also gives the executor something deterministic to compile and validate. [18]

The **capability router** should exist, but only as an advisor. Anthropic's routing pattern is useful when the input falls into distinct categories, and LangChain's LLM tool selector can reduce cost and improve focus by filtering irrelevant tools. In your system, though, neither should replace planning. The router's job is to propose a likely initial profile, confidence score, and maybe a cost/safety hint such as "probably `tabular_analysis`, read-only, local only." AG1 should still produce or adapt the plan, and `system_entry` should still own enforcement. That preserves planner primacy while letting you optimize latency and prompt size. [19]

The **data-reference resolution design** should be a first-class subsystem, not an afterthought. Every imported file should get a stable `asset_ref` and version. Spreadsheets should produce `sheet_ref`, detected table regions, normalized column IDs, row IDs, and optionally natural keys. Documents should produce `doc_ref`, page refs, chunk refs, and section refs. Then add a resolver that can take user phrases such as "the Acme row," "column revenue," "cell B12," "the second contract," or "the paragraph about termination" and map them to canonical references plus confidence. MCP's `resources/list`,

`resources/read`, URIs, templates, and subscriptions provide the right mental model for this: references should be addressable and readable on demand, not flattened into prompt text by default. Anthropic's context-engineering guidance reinforces the same idea by recommending lightweight identifiers and just-in-time loading rather than shipping whole objects into context up front. LlamaIndex's node-reference model gives a concrete example of how references can point from one representation to another, such as a chunk to its larger parent section. [20]

For ambiguous references, the resolver should return a **candidate set** with confidence and provenance rather than silently guessing. If one candidate is dominant and the action is read-only, the planner may proceed while recording the confidence and evidence. If the action is side-effectful or ambiguity is material, the run should pause for clarification or approval. This follows the broader principle from OpenAI's human-review guidance and MCP's consent model: ambiguity plus side effects should not be hidden inside a tool loop. [21]

Data workflows for web, tables, and documents

For **web search and research**, implement at least two operating modes. A **lookup mode** performs one or two searches and extracts a direct answer when the goal is narrow. A **research mode** performs iterative query generation, retrieval, page opening, note extraction, contradiction checking, and final synthesis when the task is broader or the first pass is incomplete. OpenAI's web-search guidance explicitly distinguishes fast non-reasoning search from agentic search and from deeper research modes, while Anthropic's research-system notes emphasize better performance when search starts broad, narrows progressively, evaluates source quality, and uses citations as a separate finalization pass. For your local-first system, research mode should still remain single-agent at the top level, with optional **deterministic parallel fetches** rather than autonomous spawned agents. [22]

The **web research capability** should emit structured evidence, not prose. A good observation object contains the query, result rank, URL or local mirror ID, source type, timestamp, extracted claims, and excerpt locations. A later synthesis step should operate only over these normalized observations. This is consistent with OpenAI's web-search output model, where web search calls and citations are first-class output items, and with Anthropic's citation-agent stage in Research, where claiming and citing are separated. It is also the easiest way to make GUI observability, retries, and validation deterministic. [22]

For **structured CSV/XLSX analysis**, prefer a deterministic local engine over free-form generated code. LlamaIndex's own documentation warns that its `PandasQueryEngine` and `PolarsQueryEngine` expose `eval` and are not recommended for production without heavy sandboxing. A safer pattern is: parse workbook metadata deterministically, infer tables and types, compile the user's analytical intent into a restricted internal DSL or parameterized query plan, execute that against a controlled local engine, and only use the LLM to generate the plan or explain results. If you want concrete substrate choices, DuckDB is an in-process OLAP database that can spill beyond memory and query local files directly, while Polars' lazy API explicitly defers execution and applies query optimizations. Both fit a local-first bounded-analysis runtime much better than unconstrained code generation. [23]

That **bounded structured-analysis design** should have explicit guardrails: a row budget, column budget, time budget, memory budget, default read-only mode, and an allowlist of operations such as filter, join, group, aggregate, sort, window, and write-derived-output. If AG1 wants anything outside the safe DSL, it should request escalation to sandboxed compute. In other words, "compute a margin column and write it to

an output file" can be allowed in a high-trust local workspace profile, but arbitrary Python should not be the baseline. This follows directly from the security warnings around LLM-controlled eval/code paths and from OpenAI's recommendation to keep orchestration in the harness and execution in a bounded compute layer. [24]

For **document Q&A**, use a citation-first retrieval pipeline. LlamaIndex's citation workflow is a good reference design: retrieve relevant nodes, attach citations, then synthesize an answer that is explicitly based solely on provided sources and says so when none are useful. Add recursive retrieval or small-to-big retrieval so a tiny chunk can lead you back to the larger section or page when needed for context. Also keep context tight. Anthropic's context-engineering guidance argues that context is finite, degrades as it grows, and is better assembled just in time through references and targeted retrieval than by dumping a full document into working memory. [25]

The **document evidence-backed answer design** should therefore require four things before a final answer can be marked "grounded": retrieved snippets or page spans, a claim-to-evidence map, confidence on each claim, and an explicit unsupported-claim path. If evidence is thin, the answer should say "the document does not clearly establish X from the retrieved evidence" rather than interpolating. That behavior is consistent with citation-first QA patterns and with Anthropic's emphasis on ground truth from the environment at each step. [26]

Validation, approvals, and observability

The right way to prevent **false success** is to make validation about **outcomes**, not agent self-report. Anthropic's eval guidance is very clear on the difference between the **transcript** and the **outcome**: the agent may say it completed a task, but the real question is whether the environment reflects the claimed result. For MutesHand, that means a step is not successful because AG1 says "done"; it is successful only if the validator confirms the file exists, the spreadsheet cell changed, the query result matches the schema, the answer cites supporting text, or the search evidence actually supports the claim. Anthropic also recommends grading what the agent produced rather than over-constraining the path, which fits a planner/executor system where there may be multiple valid tool sequences. [27]

A robust **grounding validation design** should combine deterministic and model-based checks. Deterministic checks should verify schemas, artifact existence, hash/version continuity, numeric recomputation, row counts, and citation offsets. Model-based checks should be reserved for rubric judgments that are hard to code, such as whether a summary overstates the evidence, whether the selected source is authoritative enough, or whether an answer omitted a materially relevant caveat. Anthropic recommends combining code-based, model-based, and human graders, and using deterministic graders where possible while calibrating LLM-as-judge behavior carefully. [27]

The **human approval, privacy, and spend gates** should be placed at four points. A **pre-run policy gate** decides whether the request can use local-only data, external web, or side effects. A **pre-tool gate** pauses before writes, deletes, shell/code execution, sensitive MCP actions, or external network egress. A **mid-run ambiguity gate** pauses when the resolver finds multiple plausible references and the next action would be consequential. A **pre-finalization gate** can optionally require approval for high-cost or high-impact outputs such as overwriting a source file or sending results elsewhere. OpenAI's guardrails guide maps cleanly onto this design with input, output, and tool guardrails plus human review for side effects; MCP independently reinforces explicit consent, visible tool exposure, and confirmation prompts. [28]

For **privacy-aware local-first design**, keep a hard boundary between what the model sees and what the runtime sees. OpenAI's agent-definition guide explicitly distinguishes conversation history from run context, and its integrations guidance says that when local or private MCP servers are connected from your runtime, your runtime should own the connection, approvals, and network boundaries. That is exactly the pattern you want: local credentials, raw files, and policy state stay in trusted runtime context; the model only sees the minimum necessary contents, references, or tool results. [29]

The **GUI observability requirements** are substantial because the frontend cannot be authoritative but still must make the system legible. At minimum, the UI should show the current plan version; active step and profile; tools currently exposed; approvals pending; budgets consumed; evidence objects gathered; citations or row/page references used; file diffs or output previews; validator results; and resumable checkpoints. OpenAI's tracing model is a useful benchmark here because it records generations, tool calls, handoffs, and guardrails as spans; LangGraph and LlamaIndex both emphasize streaming, debugging, validation, and durable workflow inspection. MCP's tool guidance also recommends clear UI indicators when tools are exposed or invoked. [30]

A good rule is that every user-visible answer should be backed by a **run artifact bundle**: the approved prompt context summary, plan IR, step results, evidence graph, validators, and final `execution_result`. That bundle is the real audit trail. The visible answer is only a projection of it. This is consistent with OpenAI's emphasis on result surfaces plus traces, and with Anthropic's recommendation to read transcripts and evaluate whether the system is actually measuring what matters. [31]

Contracts, roadmap, and risks

The system will be much easier to evolve if you formalize a small set of **contracts/documents** before implementation. The essential ones are: a **Plan IR spec**; a **Step Result schema**; a **Capability Profile schema**; a **Tool Manifest** with namespace/MCP boundaries; a **Data Reference spec** for files, sheets, rows, columns, pages, chunks, and entities; an **Evidence schema**; an **Approval Policy manifest**; a **Trace/Event schema**; and an authoritative `execution_result` **contract**. MCP's formal treatment of tools, resources, prompts, capability negotiation, and typed schemas is a good model for the level of explicitness you want. LlamaIndex's workflow validation model is also a useful reminder that typed edges and event contracts catch design errors early. [32]

The **implementation roadmap** should be shaped as a few large slices, not incremental prompt patches. **First slice:** build the deterministic control plane—`system_entry`, lifecycle state machine, plan/result contracts, trace store, evidence store, and approval engine. **Second slice:** build the local data substrate—file/document/tabular parsers, canonical references, and read-only query/runtime tools. **Third slice:** add AG1 planning with step-scoped profiles, dependency-aware Plan IR, and replan hooks. **Fourth slice:** ship the three high-value mixed workflows—document Q&A with citations, row/entity extraction plus web research, and bounded CSV/XLSX analysis plus write-back. **Fifth slice:** harden with validator suites, offline evals, regression datasets, production observability, and redaction/privacy controls. Anthropic's repeated advice to start simple and add complexity only when it demonstrably improves outcomes applies directly; OpenAI likewise recommends inspecting traces early before tuning. [33]

If you later find a real need for parallelism, the first escalation should be **deterministic parallel execution of independent acquisition steps**, not a fully autonomous subagent system. Only after you have strong evaluation, tracing, reference resolution, and evidence contracts should you consider bounded specialist

agents or orchestrator-worker fan-out. Anthropic's Research architecture shows that multi-agent systems can be excellent for breadth-first, parallelizable research, but the same post also documents coordination bugs, over-delegation, excess tool use, and sharply higher token costs. [34]

The most important **anti-patterns to avoid** are these:

- **Replacing the planner with a router.** Routing is useful, but in your system it should only suggest profiles or cost lanes, never become the execution authority. [19]
- **Tool-surface bloat and overlap.** Ambiguous, redundant tools make agent behavior worse and increase prompt/context cost. [35]
- **Treating natural-language-to-code engines as the default for tabular work.** LlamaIndex explicitly warns that eval-based dataframe tools are unsafe for production without heavy sandboxing. [36]
- **Accepting self-reported completion.** Validate against environment state, artifacts, and evidence, not the assistant's transcript alone. [27]
- **Stuffing full files and long histories into context.** Anthropic's context-engineering guidance argues for tight, just-in-time context assembly and explicit compaction or note-taking when interactions get long. [37]
- **Using hardcoded regex workflows for prompt phrases.** That fights the architecture instead of improving it; use typed reference resolution, capability profiles, and plan compilation instead. This is also consistent with Anthropic's warning against brittle, over-hardcoded prompting. [37]
- **Making the frontend authoritative.** The GUI should render projections from the run bundle; it should not own lifecycle, truth, or final state. OpenAI's harness/compute split is the right mental model here. [38]
- **Delaying evals and traceability.** Anthropic's eval guidance and OpenAI's tracing guidance both argue that reliability improves only when transcripts, outcomes, and regressions are made visible early. [39]

Taken together, the architecture I would target is: **one agent, many capability profiles, deterministic execution, first-class references, evidence-backed answers, and explicit approval/validation at every state-changing boundary**. That gives you dynamic mixed workflows without brittle hardcoded routes, while preserving the local-first, privacy-aware, planner-centric control model you specified. [40]

---

[1] [2] [7] [14] [17] [19] [33] [40] Building Effective AI Agents \ Anthropic
https://www.anthropic.com/engineering/building-effective-agents

[3] [38] Sandbox Agents | OpenAI API
https://developers.openai.com/api/docs/guides/agents/sandboxes

[4] [2210.03629] ReAct: Synergizing Reasoning and Acting in Language Models
https://arxiv.org/abs/2210.03629

[5] [11] [16] [18] Plan-and-Execute Agents
https://www.langchain.com/blog/planning-agents

[6] Orchestration and handoffs | OpenAI API
https://developers.openai.com/api/docs/guides/agents/orchestration

8 32 Specification - Model Context Protocol
https://modelcontextprotocol.io/specification/2025-06-18

9 Introduction | Developer Documentation
https://developers.llamaindex.ai/python/llamaagents/workflows/

10 Microsoft Agent Framework Overview | Microsoft Learn
https://learn.microsoft.com/en-us/agent-framework/overview/

12 13 Tool search | OpenAI API
https://developers.openai.com/api/docs/guides/tools-tool-search

15 Overview - OpenAI Agents SDK
https://openai.github.io/openai-agents-python/sessions/

20 Resources - Model Context Protocol
https://modelcontextprotocol.io/specification/2025-06-18/server/resources

21 28 Guardrails and human review | OpenAI API
https://developers.openai.com/api/docs/guides/agents/guardrails-approvals

22 Web search | OpenAI API
https://developers.openai.com/api/docs/guides/tools-web-search

23 24 36 Pandas
https://developers.llamaindex.ai/python/framework-api-reference/query_engine/pandas/?utm_source=chatgpt.com

25 26 Build RAG with in-line citations | Developer Documentation
https://developers.llamaindex.ai/python/examples/workflow/citation_query_engine/

27 39 Demystifying evals for AI agents \ Anthropic
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

29 Agent definitions | OpenAI API
https://developers.openai.com/api/docs/guides/agents/define-agents

30 Tracing - OpenAI Agents SDK
https://openai.github.io/openai-agents-python/tracing/

31 Results and state | OpenAI API
https://developers.openai.com/api/docs/guides/agents/results?utm_source=chatgpt.com

34 How we built our multi-agent research system \ Anthropic
https://www.anthropic.com/engineering/multi-agent-research-system

35 37 Effective context engineering for AI agents \ Anthropic
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents